DEVON THIELE

THE

THIELE MACHINE

A Computational Model
with Explicit Structural Cost

4 0 0

Two computers that end up the same

Put two computers on a table. Hand each of them the same list of a thousand numbers.

The first one sorts it. It compares, swaps, walks the list again and again until nothing is out of order. Then it does one more thing: it walks the whole list front to back and checks that every number is no bigger than the one after it. That check could have failed. It passes. Then it tidies up after itself, wipes its scratch work, and leaves the sorted list in memory. (That check compares neighbours, so what it settles is that the output is in order. That the output holds your thousand numbers and no others is a second claim, and it would take a second check comparing the output with the input. When I say sorted below, I mean in order.)

The second one never looks at your list. Somebody typed the sorted version into it earlier, same thousand numbers, same order, and it puts them in memory.

```
1  2  3  ...
```

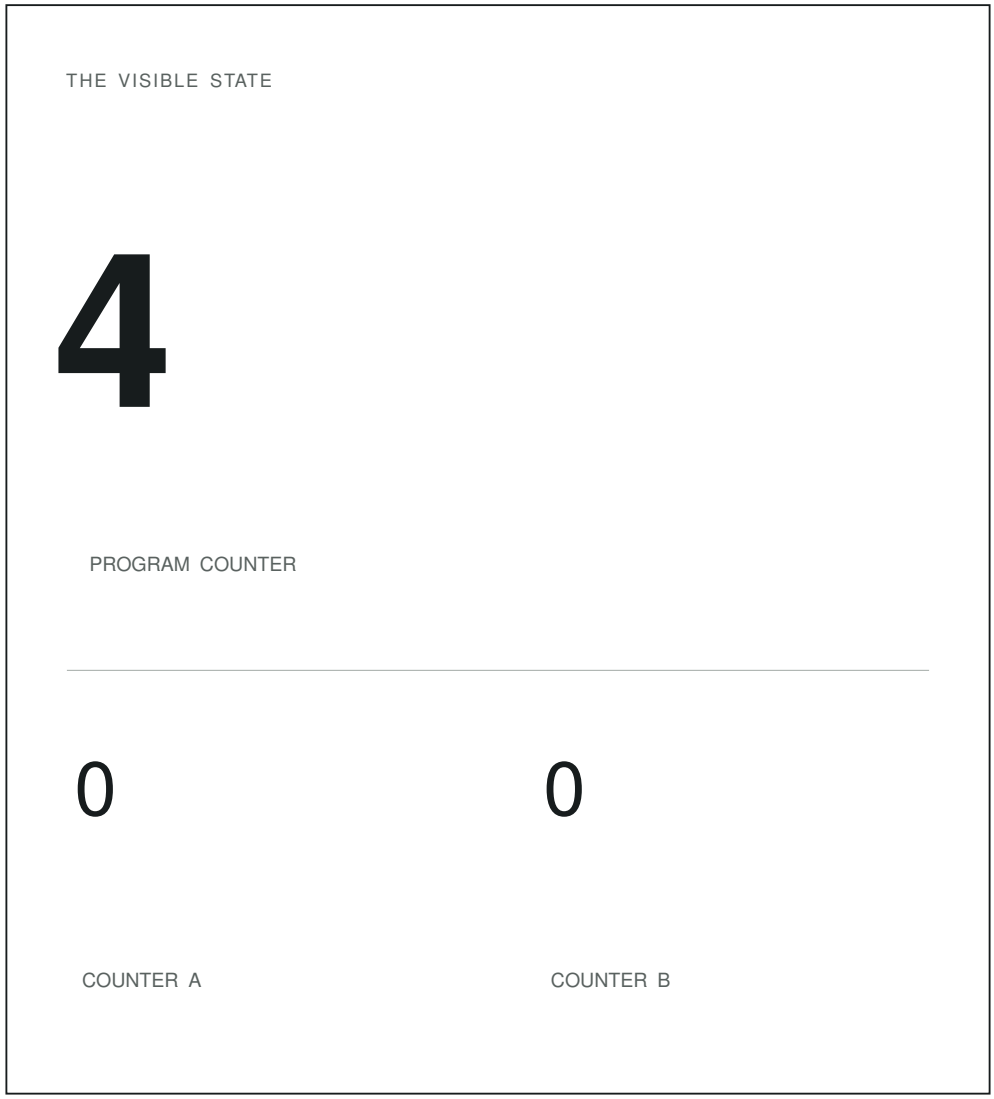
Same bits.

Now wait until both are done and look inside. Same bits in memory. Same bits on the screen. Set it up right, which you can, and it's the same everything, down to the last bit. One of them found something out about your list and stood behind it. The other one is holding identical bits and found out nothing. Wheel them out on a cart, swap them, wheel them back, and ask me which is which. I can't tell you. You can't tell me. And the part that bugs me most: built the ordinary way, neither machine can tell you either.

```
1  2  3  ...
```

What the window shows.

DEC A 0 DEC A 0 DEC A 0



What the window shows.

CHECK COMMIT CERTIFY

THE VISIBLE STATE

4

PROGRAM COUNTER

0

COUNTER A

0

COUNTER B

LEDGER 3

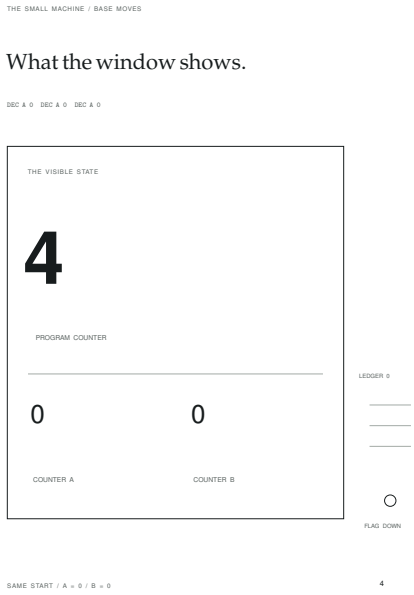




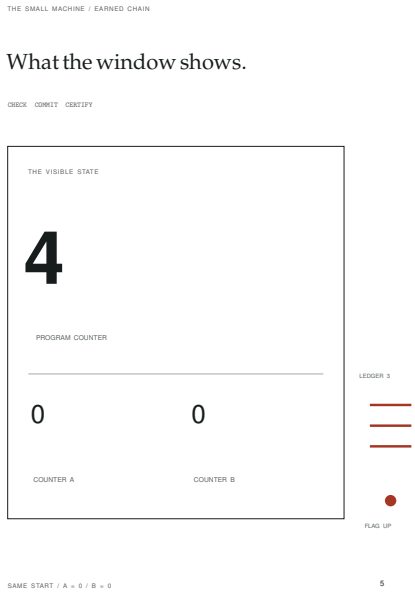
FLAG UP

Look again.

These are the two pages you just passed. The rectangle is the window; the marks beside it belong to the record.



PAGE 4

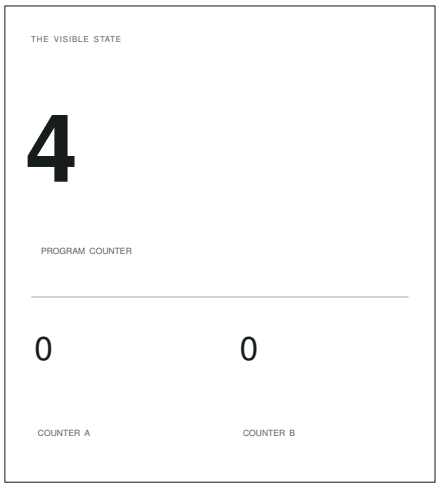


PAGE 5

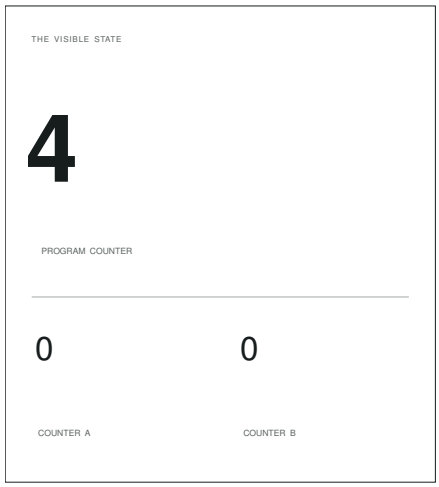
The window is unchanged. The question “is the flag up?” has two different answers.

The same view.

The two windows below are taken directly from those pages. Everything outside the rectangle has been removed.



CROP OF PAGE 4



CROP OF PAGE 5

Here's the mathematics, and I promise it's all of it. You have a set of states, the real things. You have a window, which is a function that takes a state and gives you what you can see of it: the shadow, the slice, the half-stick without notches. And you have a question about the state, which is also a function: it takes a state and gives an answer. The thing you want is a *decoder*: a rule that takes what you can see and gives the right answer to the question, every time. When does one exist?

The whole fact, in one paragraph

Think about two states that cast the same shadow. The decoder sees the same shadow for both, so it gives the same answer for both. So if the question has *different* answers on those two states, the decoder is wrong on at least one of them, and there's nothing it can do about it, because it never saw the difference. That's the whole fact. If a decoder exists, then the question must already give the same answer on every pair of states that look the same through the window. Mathematicians call the set of states that cast one particular shadow the *fibre* over that shadow (think of the shadow as a point on the wall and the fibre as the bundle of all the things that could have cast it), and the sentence is: a decoder exists only if the question is constant on every fibre. That's the theorem just below, and the "only if" is the direction that bites. The other direction, "if the question is constant on fibres then a decoder exists," is true too, but it needs you to be handed, for every possible shadow, one actual state that casts it, so you can look up the answer there, and the handing is not free either, since someone has to supply a representative for every shadow. (You can always fill a gap with a default answer. The default doesn't recover what was really there.)

The shape on the previous spread is an instance of this argument.

THE STATEMENT

Exactly. Let S be any set of states, V any set of views, $w : S \to V$ a window, and $q : S \to A$ a question with answers in any set A. A *decoder* is a function $d : V \to A$ with $d(w(s)) = q(s)$ for every state s. The question is *constant on fibres* when $w(s) = w(s')$ implies $q(s) = q(s')$.

STATE
S

VIEW
V

WINDOW
w

QUESTION
q

DECODER
d

Decoders need constant fibres

If a decoder exists, the question is constant on fibres. Conversely, if the question is constant on fibres and you are handed, for every view v, a state $r(v)$ with $w(r(v)) = v$, then $d(v) = q(r(v))$ is a decoder.

PROOF

If $w(s) = w(s')$ then $q(s) = d(w(s)) = d(w(s')) = q(s')$. Conversely, $d(w(s)) = q(r(w(s)))$, and $r(w(s))$ has the same view as s, so by constancy its answer is $q(s)$.

□

Start.

No facts on the record. No commitment. The flag is down.

THE VISIBLE STATE

1

PROGRAM COUNTER

0

COUNTER A

0

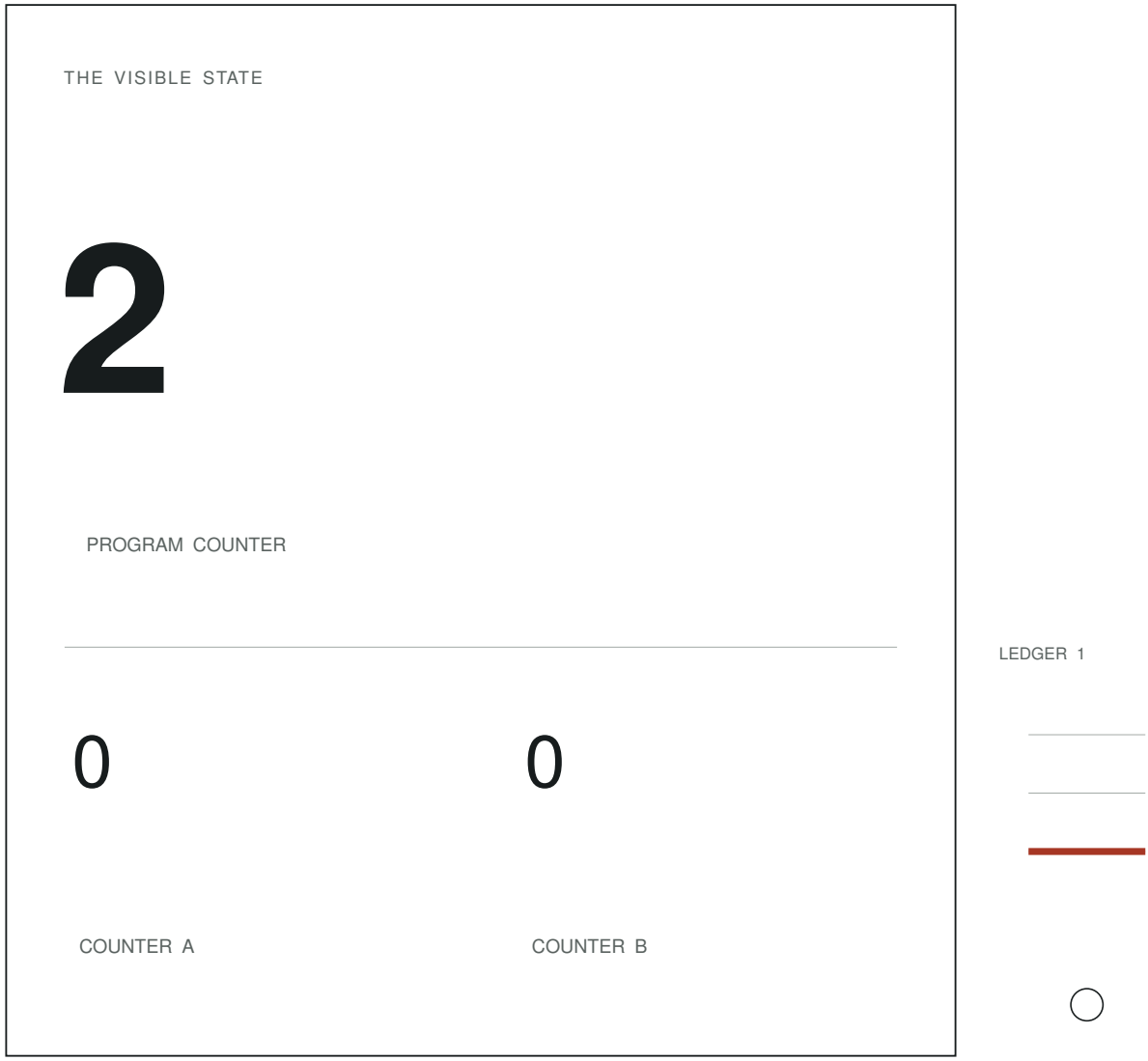
COUNTER B

LEDGER 0

FLAG DOWN

Check.

The counter is zero. The successful check records that claim about edition zero.



Commit.

The commitment names the checked claim about the same edition. The counter stays zero.

THE VISIBLE STATE

3

PROGRAM COUNTER

0

COUNTER A

0

COUNTER B

LEDGER 2



FLAG DOWN

Certify.

The flag rises. The ledger records three charged instructions.

THE VISIBLE STATE

4

PROGRAM COUNTER

0

COUNTER A

0

COUNTER B

LEDGER 3





FLAG UP

How I got here

What I am is curious, past the point of good sense.

I pick a thing up and cannot set it down until I have turned it over enough times to see how it holds together, and this time it did not let go.

I am not a programmer and I was never formally taught any of this.

I was not trying to build a formally verified CPU, and the one I came close to building later, on the way here, is not part of this book.

I would not have known what one was if it bit me.

I was trying to build a 3D renderer.

There was no plan behind it.

Just a pull to make something real with my own hands, and the kind of stubbornness that will not let a thing go until it stands up on its own.

I have worked on it in off hours every day since.

Once I started digging in, two things came clear fast. I knew nothing about computers, and the field was already saturated, every obvious road paved and crowded. So I needed an angle nobody else was standing on, and the angle I had wasn't a strategy. It was just the only way my head works. I did not know what compositional meant. I had never heard of category theory. I called the thing I was building digital DNA, because that is what it felt like in my hands, and I thought about it in the only language I had: gravity pipes and directions and vectors, things that were linear and somehow not linear at the same time, structure that flowed and joined and came apart without my owning one proper word for any of it. I built a custom rendering pipeline out of those intuitions, by feel, throwing out pieces that did not sit right even when I could not have told you the rule they were breaking. And then, the way I always do, I went looking for what I was actually doing, and found out it already had a name. An old one. A deep one. What I had been reaching for in the dark was composition. It was category theory, a whole field I had never once heard of, sitting there describing the exact shapes I had been shoving around. The moment that landed, the rendering pipeline stopped being a rendering pipeline and started turning into a categorical engine, and I started swallowing the real theory as fast as I could find the words for things I had already built.

The method underneath was crude and it held. I wrote a long structured spec in JSON (a plain-text format for describing data) as a context document, fed it to Copilot, and had it build one Python file per concept, one at a time, each with its own tests, and I would not move to the next until I could watch the current one actually run. Category. Object. Morphism. Functor. Monoidal product. It came together in pieces and never in chapters, the way a picture resolves out of static. The Yoneda lemma (a result buried deep in category theory) was the piece that made the rest snap into place, the night I quit borrowing other people's runtimes and started writing my own around it, with Z3 (a constraint solver out of Microsoft Research) wired into the middle.

The ink stayed.

This is the cover again, unchanged. Its numbers are the final window of the earned run: program counter 4, counters 0 and 0. The three marks beside the frame are its ledger. The filled circle is its raised flag.



Now there's you.

The theorems are what they are, with their premises printed. The interpretation is mine, and I've tried to say at every turn which is which, because the first time through I was the only one checking, and I've met me, and that's not enough people. Now there's you.

THE THIELE MACHINE

Devon Thiele

What this tests

The book becomes an object in its own argument. Pages 6 and 7 contain actual reproductions and crops of pages 4 and 5. The restricted views are identical, while the records outside them differ. The reader can inspect that claim on the printed object.

The form becomes readable as the argument develops. The cover carries the final state of a specific machine run. The sequence teaches the reader to read its frame, numbers and marks. When the cover returns, its ink is unchanged.

Page turns expose real transitions. The four states on pages 10–13 come from the small machine's CHECK, COMMIT, CERTIFY run. The marks count those instructions. They do not measure the reader's attention, understanding, reading effort, or physical energy.

Print and PDF use the same pages. The mechanism uses position, repetition, comparison and page turning. Color is supplementary: line weight, count and filled or empty circles carry every distinction in monochrome too.

This is a prototype. The selected prose and theorem text come from the released manuscript. The demonstration captions are new. This study tests a design language; it is not a newly published edition or a claim that the entire book has been redesigned.

What comes from where

Manuscript. The opening's two computers; Chapter 7's window argument, statement and proof; Chapter 1's autobiographical opening; and the conclusion are excerpts from Devon Thiele's *The Thiele Machine*. Their order here is a design sample, not a proposed rearrangement of the manuscript.

Machine. From $(A, B) = (0, 0)$, three zero-branch DEC instructions and the earned CHECK, COMMIT, CERTIFY chain both reach the window $(4, 0, 0)$. Their ledgers are 0 and 3; their flags are down and up. The generator executes both programs using `thiele_small/small.py`, the handwritten Python mirror, and checks the displayed states.

Mathematics. The window theorem is the manuscript's theorem "Decoders need constant fibres." Its checked counterparts are `decoding_requires_fiber_constancy` and `selected_representatives_give_decoder`, in `ObservationPolicy.v`.

Earlier design work. The recorded design in PR 205 and its reversal in PR 206 were inspected, including the generated case diagrams and the prior PDF. Private brainstorming conversations are not present in the repository.

Production. Pages are 190 by 254 millimeters. Text and diagrams remain vector objects, fonts are embedded, and the same PDF can be read on screen or printed at actual size. The single-page PDF uses a facing-page viewing preference. No special paper, transparency, fold, animation or viewer behavior is needed to understand the sequence.